Virtual Memory Replacement & Allocation: Progress Report

Pranav Kumar Kaliaperumal, Akanksha Gutal, Spandana Erukulla
Department of Computer Science and Engineering
CSCI 5573: Operating Systems
University of Colorado Denver

***Abstract*—This progress report, current as of April 4, 2026, outlines the development of a virtual memory simulator and benchmarking framework created to analyze page replacement and frame allocation in environments running multiple programs. The simulator and experimental tools now support FIFO, LRU, NRU, Second Chance, and WSClock replacement algorithms, as well as global, local, and proportional allocation strategies. Workload drivers generate both synthetic and trace-based scenarios. The system streamlines parameter exploration, logs data on page faults and performance slowdowns using isolation-based tests, and outputs results as CSV and JSON files, along with visual plots. We have run the framework on both synthetic and trace-driven workloads, carrying out initial experiments including stress tests at the edge of total working-set memory. As we approach the final submission, we are broadening the evaluation (adding more workloads, traces, repetitions, and sensitivity analyses), refining key figures and tables and their explanations, and improving the packaging and documentation for reproducibility.**

I. Core Tables and Figures

This progress report is centered on a concise set of core evidence—three to four figures and three to four tables—that will be refined in the next phase for the final submission. Additional materials found in `results/` serve as supplementary resources.

A. Core Figures (Current Candidates)

For the final submission, we have identified a core set of candidate figures to visually communicate our primary findings, all currently generated and stored within the `results/figures/` directory. These include an evaluation of efficiency under memory pressure (**F1**: `page_faults_vs_memory_pressure.png`), an analysis of the worst-case fairness cost associated with different policies (**F2**: `worst_case_slowdown_vs_policy.png`), and a broader comparison of Jain's fairness index across allocation strategies (**F3**: `jains_fairness_vs_policy.png`). Additionally, we provide a trace-driven cross-check (**F4**: `trace_driven_comparison.png`) to validate our synthetic results against real-world access patterns. Should page limits allow, we also plan to include an auxiliary figure (`slowdown_variance_vs_memory_pressure.png`) to further illustrate the dispersion of fairness metrics.

B. Core Tables (Current Candidates)

Similarly, we have curated a set of core tables to summarize our quantitative data. The complete experimental matrix (**T1**) detailing version-locked factor levels and repetition counts is mirrored in Section VI-A and available in `docs/final_experiment_matrix.md`. Headline results outlining the best-performing policies for each workload are captured in **T2** (`results/tables/best_policy_by_workload.md`). The formal evaluation of our three research hypotheses (H1, H2, and H3) is synthesized in **T3** (`results/tables/hypothesis_summary.md`). Finally, **T4** offers a compact statistical summary—extracted from `final_aggregate_results.csv`—that focuses on the near-WSS pressure point, detailing means, standard deviations, confidence intervals for faults and slowdowns, as well as Jain's index and the coefficient of variation.

II. Executive Summary and Project Overview

Virtual memory helps computers use their resources more efficiently by letting programs work with more memory than what is physically available, a requirement increasingly vital in big memory servers and flat page table architectures [1], [2]. When there's not enough physical memory to go around, the operating system has to choose which pages to remove and how to split up the available memory between different programs. Modern memory allocation, NVRAM integration, and compaction techniques all navigate these constraints [3], [4]. These choices have a big impact on how often page faults happen, how much disk activity is generated, and how fairly resources are shared among running programs.

Our project looks at how page replacement and frame allocation work together, not just at replacement strategies on their own, reflecting modern trends that treat cache replacement and memory allocation as joint problems [5], [6]. Earlier research, like Belady's, showed that basic approaches such as FIFO can fail in certain situations, while Denning's review highlighted the need for policies that keep data close to where it's needed and can adjust as programs change [7], [8]. We explored whether these well-known rankings still hold up or start to break down when several programs are fighting for scarce memory.

a) Current Status.: At this point, the project has moved from just an idea to a working experimental setup that lets us evaluate both page replacement methods and frame allocation strategies using synthetic and trace-based workloads. The system can now run FIFO, LRU, NRU, Second Chance, and WSClock algorithms, and tries out global, local, and proportional ways to divide up memory. It automatically measures page faults, disk usage, slowdowns, and fairness, creating tables, summaries, and visualizations ready for analysis. Early results show that our framework can produce reliable comparisons and highlight real trade-offs between efficiency and fairness. So far, the evidence supports our first two hypotheses (H1 and H2), but the third (H3) seems weaker with the current data. These results are still early, so we'll treat them as provisional until the final round of analysis and reporting is done.

III. System Architecture

A. Software Architecture

The software architecture of our custom virtual memory simulator is built around three primary, interacting modules. The **Workload Generator** is responsible for parsing real-world CSV traces and synthesizing various memory access patterns (such as high-locality, streaming, and thrashing) to drive the simulation. Operating alongside it is the **Memory Manager**, a core component that handles the mapping of virtual to physical memory, manages the chosen frame allocation strategies (Global, Local, and Proportional), and maintains the page tables for multiprogrammed workloads. Finally, the **Policy Engine** encapsulates the logic for making page replacement decisions, actively implementing the FIFO, LRU, NRU, Second Chance, and WSClock algorithms.

B. Hardware Architecture

The simulator models a target hardware system featuring a single-tier storage hierarchy with configurable physical frames, allowing us to accurately simulate varying degrees of memory pressure. For execution, all experiments and benchmarks were run locally on a host machine equipped with a 12th Gen Intel Core i5-12400F processor (2.50 GHz, 6 cores), 16 GB of RAM, and an NVIDIA RTX 3050 GPU. To minimize host-OS interference and ensure measurement stability for wall-clock slowdown metrics, we utilized wrapper scripts to pin processes to specific CPU cores.

IV. Research Questions and Hypotheses

Our study centers on three main research questions:

A. RQ1: Efficiency vs. Fairness

What pairings of page replacement and frame allocation can keep page faults low without sacrificing fairness between programs? We thought that LRU with global allocation would usually lead to fewer faults when memory is tight and locality is high, but that it might also make some processes less fairly treated.

B. RQ2: Memory-Pressure Thresholds

Is there a clear point where global allocation stops improving throughput and instead hurts fairness? We expected this shift to happen around the total working-set size, where even a small drop in available memory could suddenly cause much more interference between programs.

C. RQ3: Simpler Policies Under Heterogeneous Workloads

Can straightforward strategies like FIFO or NRU do better than more complex, history-based policies when fairness matters most? We thought that in some mixed workloads, pairing simple policies with local allocation could actually be better for fairness, even if they didn't always minimize total page faults.

V. Operational Definitions and Current Measurement Support

In our project, we defined fairness mainly by looking at how much each process slowed down compared to when it ran by itself. To measure this, we recorded the actual runtime for each process running alone and then again when running with others, using `time.perf_counter_ns()`. We calculated slowdown as $\text{slowdown} = t_{\text{shared}}/t_{\text{isolated}}$. To sum up fairness, we looked at Jain's index, how much slowdown varied, the coefficient of variation, and the range between the slowest and fastest processes.

To analyze working sets, we used a sliding-window method to estimate the working-set size (WSS) for each process. This let us get a good idea of the total WSS, which we used as a baseline in our memory-pressure experiments. Alongside our fairness and slowdown measurements, the simulator also kept track of overall page faults, page replacements, disk reads and writes, and total disk activity. Since wall-clock slowdown can be pretty noisy—especially for processes that don't run very long—we ran a campaign to check how stable our isolation baselines were and included statistics like the coefficient of variation (CV) in our artifact bundle.

VI. Methods and Setup (Required Content Audit)

This section summarizes the current experiment matrix, trace support, slowdown measurement method, and the environment controls used in the present progress-stage campaign. We use a custom Python-based virtual memory simulator with trace-driven replay support and benchmark-control wrappers for repeatable experiments.

A. Current Experiment Matrix

The current matrix evaluates FIFO, LRU, NRU, Second Chance, and WSClock under GlobalAllocation, LocalAllocation, and ProportionalAllocation. Synthetic workload families include high_locality, streaming, mixed, and thrashing. The current progress-stage sweep also uses memory levels of 0.8x, 1.0x, 1.2x, and 1.5x aggregate WSS, process counts of 2, 4, and 8, and 10 repetitions per configuration.

Trace-driven replay is supported through the same runner used for synthetic experiments. The current trace set covers sample, locality-heavy, streaming, mixed, and thrashing

families, with a smaller demo-only trace retained for artifact packaging.

B. Environment Controls and Slowdown Measurement

Benchmark runs record environment metadata and use wrapper scripts to reduce host noise. In practice, we rely on quiet-run and pinned-core wrappers to stabilize runs, capture environment details, and make the benchmark conditions explicit.

For slowdown, each workload is executed both in a shared multiprogrammed run and in isolated per-process runs. We record runtimes with `time.perf_counter_ns()` and compute slowdown as $t_{\text{shared}}/t_{\text{isolated}}$. Fairness summaries are then derived from the slowdown distribution. In the current progress-stage data, these slowdown measurements are useful for directional comparisons, but the wall-clock component remains noisier than the page-fault and fairness-pattern trends.

VII. Experimental Design

The framework is organized around synthetic workloads representing high-locality, streaming, mixed, and thrashing access patterns (the current version-locked set), plus additional diagnostic patterns (e.g., random and loop) used during development. Single-process and multi-process experiment runners automate repeated trials across combinations of algorithms, allocation policies, and memory sizes. The experiment scripts supported:

- single-process comparisons across all implemented replacement algorithms;
- multi-process comparisons between global and local allocation;
- memory-pressure sweeps over several frame counts; and
- fairness-oriented workload mixes designed to stress policy trade-offs.

Besides synthetic workloads, our repository also supports running experiments on real-world traces. We convert trace CSV files into a common event format so they can use the same runner as our synthetic tests. We made sure to test both this trace-driven workflow and our isolation-baseline setup (see Section X). For reproducibility and to cut down on system noise, we used wrapper scripts to log all environment details and pin runs to specific CPU cores. We've included a baseline stability report in our artifacts so others can see how much run-to-run variation there was.

VIII. Performance Measures

The simulator keeps track of page faults, page replacements, disk reads and writes, and total disk activity for every run. These stats were enough for us to compare different replacement strategies and see the main effects of memory pressure. We also built in tools for things like variance, confidence intervals, and t-tests to help analyze results from repeated tests.

We checked fairness by looking at how slowdown was spread out among the different processes, using Jain's index and other measures of variation. We handled everything from running shared and isolated tests, to collecting results, to plotting them. Still, we considered these fairness numbers a bit noisy, especially for workloads that didn't run for very long. Even with pinned-core runs, our baseline stability tests showed that some setups had more noise than others.

IX. Methodological Controls

We built some basic quality controls right into the code. Experiments could be repeated, workloads could be run with the same random seed, and the process was automated so we could re-run the same test setups and get consistent results. This helped avoid random changes from one run to the next.

We also wrote scripts to help run benchmarks more reliably by letting us pin runs to specific CPU cores if needed, and by saving detailed environment snapshots in `results/metadata/`. To measure any leftover noise, we put together a baseline stability report that grouped similar runs and calculated how much their results varied. At this point, the biggest remaining limitations are less about missing features and more about how far we want to push the campaign—like adding more sensitivity checks, running tests longer, or including a wider variety of traces before wrapping up the project.

X. Preliminary Results and Current Findings

The current cleaned campaign snapshot covers the full synthetic matrix described in Section VI-A together with five report-grade traces. The locked aggregate includes the main workload families, all three allocation policies, all five replacement algorithms, and repeated runs across the pressure settings used for the current study. The embedded tables and figures below are drawn from that cleaned snapshot.

At this stage, the current findings still point in the same general direction as the earlier progress-stage read: H1 appears to be supported, H2 is likely to hold near the working-set boundary, and H3 is not currently supported by the mixed-workload snapshot. In other words, the present campaign continues to suggest a real efficiency-versus-fairness trade-off under pressure, with global allocation helping system-level efficiency in some settings while worsening fairness-sensitive outcomes.

The main caveat is measurement stability. Slowdown is based on wall-clock shared-versus-isolated timing, and the pinned-core baseline study showed that isolation timings can still be noisy for some configurations. For that reason, the current findings should be read primarily as directional evidence, while page-fault trends and broader fairness patterns remain the more stable signals in this progress report.

XI. Embedded Evidence Snapshot

To make things easier for readers, we now include a snapshot of key data and visuals right here in the report instead of just pointing to files. The tables below come straight from our main campaign data, and the figures show exactly what's in the headline results.

TABLE I
CLEAN CANONICAL CAMPAIGN SCOPE USED FOR THE PROGRESS REPORT.

Factor	Clean canonical setting
Algorithms	FIFO, LRU, NRU, Second Chance, WSClock
Allocation policies	GlobalAllocation, LocalAllocation, ProportionalAllocation
Synthetic workloads	high_locality, streaming, mixed, thrashing
Memory levels	0.8x, 1.0x, 1.2x, 1.5x aggregate WSS
Process counts	2, 4, 8
Repetitions	10
Report-grade aggregate	1005 config groups / 10,050 trial rows
Trace families	Five report traces; one demo-only trace is excluded from report-grade aggregation

TABLE II
HEADLINE RESULTS BY SYNTHETIC WORKLOAD IN THE CLEANED CANONICAL AGGREGATE.

Workload	Lowest page faults	Lowest worst slowdown	Highest Jain's index
high_locality	FIFO + GlobalAllocation (144.0)	NRU + ProportionalAllocation (1.5105)	SecondChance + LocalAllocation (0.99997)
mixed	FIFO + GlobalAllocation (202.7)	NRU + LocalAllocation (1.5054)	FIFO + LocalAllocation (0.99988)
streaming	FIFO + GlobalAllocation (340.0)	NRU + LocalAllocation (1.4488)	LRU + ProportionalAllocation (0.99991)
thrashing	FIFO + GlobalAllocation (204.0)	NRU + ProportionalAllocation (1.5057)	FIFO + LocalAllocation (0.99987)

TABLE III
HEADLINE RESULTS BY TRACE WORKLOAD IN THE CLEANED CANONICAL AGGREGATE.

Trace workload	Lowest page faults	Lowest worst slowdown	Highest Jain's index
report_locality_heavy	FIFO + GlobalAllocation (4.0)	FIFO + ProportionalAllocation (1.0782)	NRU + GlobalAllocation (0.99867)
report_mixed_heterogeneous	FIFO + GlobalAllocation (11.0)	FIFO + ProportionalAllocation (0.9630)	SecondChance + GlobalAllocation (0.99846)
report_streaming_sequential	FIFO + GlobalAllocation (16.0)	FIFO + ProportionalAllocation (1.0486)	SecondChance + GlobalAllocation (0.99965)
report_thrashing_pressure	FIFO + GlobalAllocation (10.0)	FIFO + ProportionalAllocation (1.0739)	WSClock + LocalAllocation (0.99963)
sample_trace	FIFO + GlobalAllocation (5.0)	FIFO + ProportionalAllocation (0.9342)	LRU + LocalAllocation (0.99781)

TABLE IV
H1/H2/H3 SUMMARY FROM THE CLEANED CANONICAL HYPOTHESIS EVALUATION.

Hyp.	Test condition	Baseline → cmp.	Change	Status
H1	High locality, 4 procs, near 1.2× WSS	326.2 → 325.1 faults	-0.34%	Supported
H2	Aggregate-WSS boundary, 4 procs	0.01190 → 0.01886 variance	$+58.53\%$	Supported
H3	Mixed workload, 4 procs, near WSS	2.6449 → 3.2209 slowdown	$+21.78\%$	Not sup.

XII. COMPLETED WORK

To date, we have successfully implemented a suite of page replacement policies—namely FIFO, LRU, NRU, Second Chance, and WSClock—alongside global, local, and proportional frame-allocation strategies. To drive our evaluations, we developed synthetic workload generators capable of producing locality-heavy, streaming, random, mixed, loop, and thrashing access patterns. The simulation framework is further supported by automated experiment runners designed for repeated single-process and multi-process studies, as well as analytical helpers for generating summary statistics and comparing policy performance. Finally, we have established comprehensive documentation and a report skeleton structured around our three core research questions.

Our original proposal described an emulator-based experimental plan; however, during implementation we refined the methodology and built a custom Python-based virtual memory simulator and trace-driven benchmarking framework instead. This change gave us tighter control over page replacement, frame allocation, workload generation, and measurement instrumentation while still supporting the same core research questions and evaluation goals. We also double-checked the repository for any problems with running the code or mismatches in our reporting. This review turned up a few practical issues that we had to fix before we could create the current campaign snapshot. These included cleaning up some runtime bugs in the allocation path, making sure WSClock worked correctly under eviction pressure, and making sure our documentation matched what was actually measured and reported.

XIII. LIMITATIONS AND FUTURE WORK

a) Limitations.: We can measure slowdown by comparing runs done in isolation to those done with shared workloads, but the results are still a bit noisy. That means our current findings are best used to spot overall trends in policy performance at different memory-pressure levels, not as final, universally-applicable conclusions. Our current set of traces is realistic, but it's not as broad as it could be, so the most reliable takeaways will come after we finish the expanded campaign and not just from this early snapshot.

b) Next Phase.: For the next phase, we're aiming to turn our benchmarking framework into a study that's ready for a conference. This means we'll grow and test the set of trace-based evaluations, make slowdown measurement more stable and robust against noise, and finish the full statistical analysis over all repeated runs. We'll also wrap up the hypothesis summaries for H1, H2, and H3, pull our best figures and tables into the main paper, and make sure our conclusions are based on real data from our experiments. To finish, we'll put together a polished reproducibility package with all the key results, traces, scripts, and demo materials so the project stands as a complete, shareable study fit for submission.

In the next phase, we also plan to add an explicit efficiency cost model so that policies can be compared using a single composite system-level metric rather than only separate page-fault, I/O, slowdown, and fairness measures. The goal is to define a weighted cost function that combines page faults, disk reads, disk writes, replacement activity, and slowdown-related penalties, then use that model to compare whether the policies that minimize page faults also minimize overall execution cost. This extension will make the final study stronger by giving us a clearer way to discuss efficiency trade-offs alongside fairness.

XIV. REPRODUCIBILITY AND ARTIFACTS

All quantitative statements in Section X are backed by checked-in outputs located within the `results/` directory, with a comprehensive artifact index and reproduction guide available in `docs/report_artifacts.md`. The experimental bundles were generated using our primary matrix runner, `experiments/run_all_experiments.py`, executed with the `--final` flag. These runs utilized trace inputs such as `sample_trace.csv`, `demo_locality.csv`, and various `report_*.csv` files stored under `workloads/traces/`.

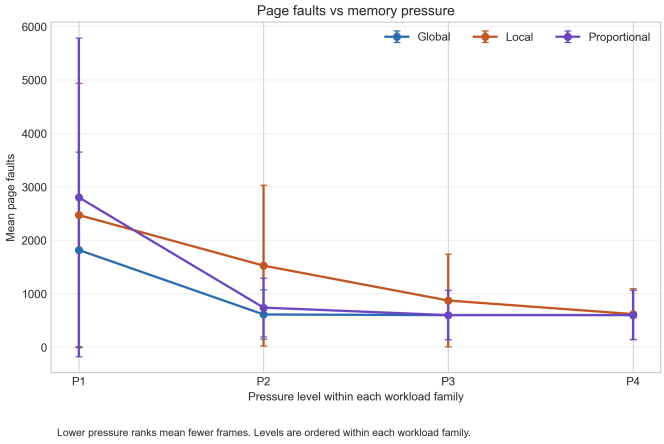

Fig. 1. Page faults vs. memory pressure in the cleaned canonical aggregate.

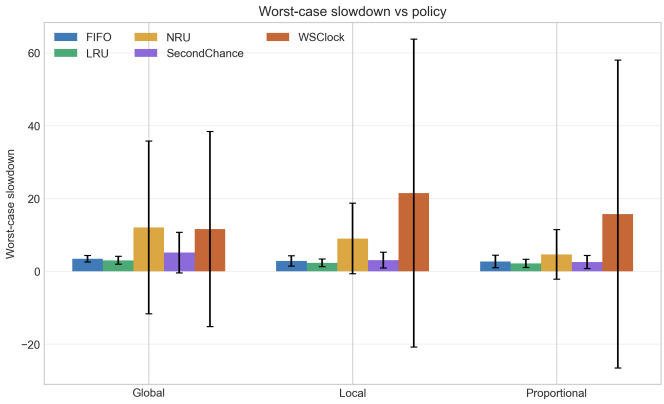

Fig. 2. Worst-case slowdown vs. allocation policy in the cleaned canonical aggregate.

The resulting raw outputs, including `shared_results.csv` and `isolation_results.csv`, are preserved in the canonical campaign folder within `results/raw/`. These raw data were subsequently refined into processed outputs—namely `slowdown_metrics.csv`, `final_aggregate_results.csv`, and `hypothesis_evaluation.csv`—located under `results/processed/`. For analysis and visualization, presentation outputs such as Markdown tables and figures are automatically generated and saved to `results/tables/` and `results/figures/`, respectively.

The framework's core analysis entry points include `main.py`, alongside specialized scripts for baseline isolation (`scripts/run_isolation_baselines.py`) and report generation (`scripts/build_report_tables.py` and `scripts/build_report_figures.py`). Data validation is strictly handled by `scripts/aggregate_final_results.py` and `scripts/validate_results.py`. Finally, to ensure stable execution and reliable artifact capture, we employed wrapper scripts (`scripts/run_quiet_benchmark.ps1` and `scripts/run_pinned.ps1`) that log environment

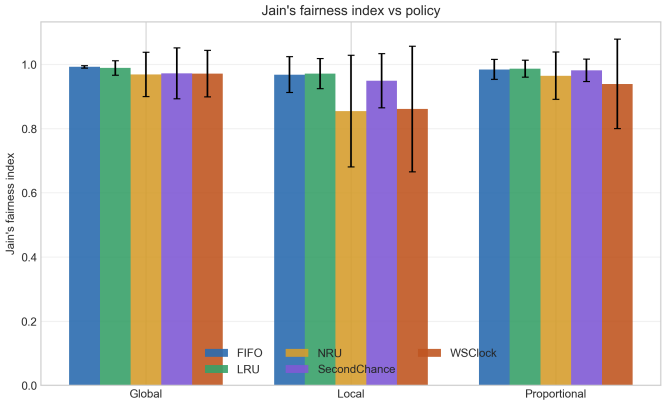

Fig. 3. Jain's fairness index vs. allocation policy in the cleaned canonical aggregate.

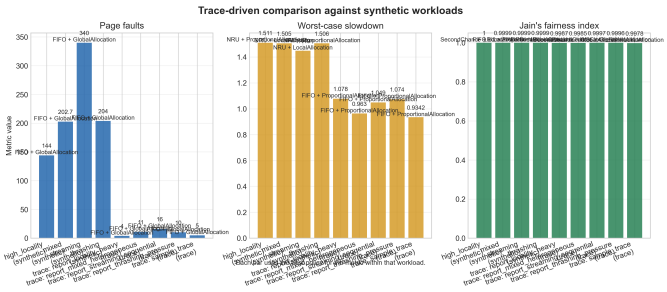

Fig. 4. Trace-driven comparison across the cleaned canonical trace families.

metadata under `results/metadata/`, while a dedicated generator (`results/demo/generate_demo_bundle.py`) compiles the final demo bundle as documented in `docs/final_demo_bundle.md`.

XV. Roles and Milestones

- **Pranav Kumar Kaliaperumal:** Led replacement-policy experiments, automation, and data collection. **Next phase ownership:** Will handle any final sensitivity checks judged worth adding and package and polish the final demo bundle.
- **Spandana Erukulla:** Led allocation-policy comparisons, workload construction, stress testing, and fairness-oriented experiments. **Next phase ownership:** Will co-handle any final sensitivity checks judged worth adding beyond the cleaned canonical campaign.
- **Akanksha Gutal:** Led analysis/validation, visualization, statistics, and report integration. **Next phase ownership:** Will lead result validation, finalize the compact figure/table set, and polish the final report and artifact checklist.
- **All Team Members:** Will cross-review conclusions and complete a fresh-run reproducibility check before submission.

a) Milestones (as of April 4, 2026).: **Completed**

- Core virtual-memory simulator and experiment harness (replacement + allocation) working end-to-end.

- Algorithm implementations: FIFO, LRU, NRU, Second Chance, WSClock; plus global/local/proportional frame allocation.
- Trace-driven replay integrated alongside synthetic workload generation.
- Runtime-based slowdown measurement pipeline (isolation baselines + shared runs) with logged CSV/JSON outputs.
- Wrapper scripts for stable execution and artifact capture (quiet runs, pinned cores, environment metadata).
- Clean canonical campaign rebuild and report-grade aggregation (1005 config groups / 10,050 trial rows).

In Progress

- Final statistical interpretation and result sanity checks (baseline stability, variance/CI reporting, and robustness discussion) **(Expected: April 15, 2026)**.
- Final figure/table generation for the core submission evidence set **(Expected: April 18, 2026)**.
- Report polishing (tightening narrative, limitations, and reproducibility notes) **(Expected: April 22, 2026)**.

Remaining for Final Submission

- Final hypothesis write-up polish (H1–H3: concise interpretation aligned to the core figures/tables and presentation narrative) **(Expected: April 25, 2026)**.
- Final demo/report artifact polish (clean submission packaging around the existing demo bundle and report assets) **(Expected: April 28, 2026)**.
- Final artifact verification (fresh-run reproducibility, file-manifest consistency, and one last validation-script pass before submission) **(Expected: May 2, 2026)**.

In Progress

- Final statistical interpretation and result sanity checks (baseline stability, variance/CI reporting, and robustness discussion).
- Final figure/table generation for the core submission evidence set.
- Report polishing (tightening narrative, limitations, and reproducibility notes).

Remaining for Final Submission

- Final hypothesis write-up polish (H1–H3: concise interpretation aligned to the core figures/tables and presentation narrative).
- Final demo/report artifact polish (clean submission packaging around the existing demo bundle and report assets).
- Final artifact verification (fresh-run reproducibility, file-manifest consistency, and one last validation-script pass before submission).

XVI. Environment

This project is implemented as a Python-based virtual memory simulation and benchmarking framework. Experiments are executed through the custom simulator, with trace-driven replay support and host-side benchmark wrappers for pinned and quiet runs. The implementation relies on NumPy, SciPy, Matplotlib, and `tqdm`, and it runs from the command line on Windows, macOS, or Linux as long as the required Python environment is available.

To meet performance and reproducibility requirements, experiments were executed on a host machine equipped with an Intel Core i5-12400F processor (2.50 GHz, 6 cores), 16 GB of RAM, and an NVIDIA RTX 3050 GPU. We utilized pinned CPU cores to minimize host-OS interference during runtime isolation baselines. Building the PDF version of this report additionally requires a working local LaTeX installation.

References

[1] A. Basu, J. Gandhi, J. Chang, M. D. Hill, and M. M. Swift, "Efficient virtual memory for big memory servers," in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, 2013.

[2] C. H. Park, I. Vougioukas, A. Sandberg, and D. Black-Schaffer, "Page tables: Keeping them flat and hot (cached)," arXiv preprint arXiv:2012.05079, 2020.

[3] D. Schwalb, T. Berning, M. Faust, M. Dreseler, and H. Plattner, "nvm malloc: Memory allocation for nvram," 2015, [Online]. Available via the ADMS 2015 archived PDF.

[4] B. Powers, D. Tench, E. D. Berger, and A. McGregor, "Mesh: Compacting memory management for c/c++ applications," arXiv preprint arXiv:1902.04738, 2019.

[5] S. Ghandeharizadeh, S. Irani, and J. Lam, "Cache replacement with memory allocation," in *Proceedings of the 17th Workshop on Algorithm Engineering and Experiments (ALENEX)*, 2015, pp. 1–9.

[6] H. Noor, H. Khan, I. U. Din, M. I. Tariq, M. N. Amin, and M. Fatima, "Virtual memory management techniques," in *Securing the Digital Realm: Advances in Hardware and Software Security, Communication, and Forensics*, M. Arif, M. A. Jaffar, O. Geman, and W. Abbasi, Eds. Boca Raton, FL, USA: CRC Press, 2025, pp. 126–137.

[7] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[8] P. J. Denning, "Virtual memory," *ACM Computing Surveys*, vol. 2, no. 3, pp. 153–189, Sep. 1970.